SERVICIO NACIONAL DE APRENDIZAJE — SENA

CENTRO DE TELEINFORMÁTICA Y PRODUCCIÓN INDUSTRIAL

Regional Distrito Capital / Regional Cauca • Modalidad Virtual

EVIDENCIA GA4-220501095-AA2-EV06

TALLER: ARQUITECTURA DE SOFTWARE

Fundamentación Teórica, Incorporación de Patrones de Diseño, Vista de Componentes y Vista de Despliegue

PROGRAMA DE FORMACIÓN:
Tecnólogo en Análisis y Desarrollo de Software (ADSO)

FICHA DE CARACTERIZACIÓN:
220501095

FASE DEL PROYECTO:
Fase 2: Planeación (Guía de Aprendizaje 4)

COMPETENCIA:
220501095 - Diseñar la solución de software

APRENDIZ:
Johan Sebastián

INSTRUCTOR DE ÁREA TÉCNICA:
Instructor Técnico SENA

PROYECTO FORMATIVO DE REFERENCIA:
Sistema POS y Facturación Electrónica - Señor Hornado

FECHA Y VERSIÓN:
Octubre de 2026 • Versión 1.0 (Final)

Servicio Nacional de Aprendizaje SENA — Dirección de Formación Profesional Integral

Documento Técnico de Entrega de Evidencia de Aprendizaje

TABLA DE CONTENIDO

1. Introducción y Contexto en la Fase de Planeación

1.1 Justificación y Relevancia Formativa

1.2 Objetivos del Taller (General y Específicos)

1.3 Sistema de Referencia Tecnológica (Caso Señor Hornado)

2. Desarrollo del Taller: Preguntas Orientadoras

2.1 Pregunta 1: ¿Qué se Entiende por Arquitectura de Software?

2.2 Pregunta 2: ¿Cuál es su Función en un Sistema de Software?

2.3 Pregunta 3: ¿Cómo se Elabora la Arquitectura de Software? (Método ADD)

2.4 Pregunta 4: ¿Cómo Lograr una Buena Arquitectura? (Patrones y Principios SOLID)

2.5 Pregunta 5: ¿Cuáles son los Elementos de Diseño de una Arquitectura?

3. Vistas Arquitectónicas del Sistema (Criterios de Evaluación)

3.1 Vista de Componentes (Fundamentación y Modelado UML)

3.2 Vista de Despliegue (Justificación y Topología Física/Nube)

3.3 Decisiones de Infraestructura y Resiliencia Operativa

4. Matriz de Cumplimiento de Lista de Chequeo (IE-GA4-220501095-AA2-EV06)

5. Conclusiones Técnicas

6. Referencias Bibliográficas (Normas APA 7ma Edición)

1. INTRODUCCIÓN Y CONTEXTO EN LA FASE DE PLANEACIÓN

1.1 Justificación y Relevancia Formativa

En el marco del programa Tecnólogo en Análisis y Desarrollo de Software (ADSO) del Servicio Nacional de Aprendizaje (SENA), la **Guía de Aprendizaje 4 (GA4)** sitúa al aprendiz en la fase crucial de **Planeación**, orientada a la competencia **220501095: Diseñar la solución de software de acuerdo con los requerimientos y modelos de diseño**.

Habiendo superado las etapas de especificación de requisitos mediante casos de uso, historias de usuario, modelos de dominio e informes de factibilidad económica (fases previas del ciclo formativo), la ingeniería de software exige responder a una pregunta estructural ineludible: *¿Cómo se orquesta el sistema informático en su conjunto para garantizar que satisfaga los rigurosos atributos de calidad y soporte el crecimiento del negocio a lo largo del tiempo?*

Construir una solución de software sin una arquitectura formalmente planificada es análogo a construir un rascacielos sin planos estructurales ni cálculos de cargas dinámicas: el colapso operativo, la degradación de tiempos de respuesta, las brechas de seguridad y la imposibilidad de mantenimiento serán consecuencias inevitables. Este informe técnico desarrolla la evidencia **GA4-220501095-AA2-EV06: Taller. Arquitectura de software**, dando respuesta profunda y argumentada a los fundamentos de la disciplina.

1.2 Objetivos del Taller

Objetivo General: Elaborar un informe técnico argumentado que aborde en profundidad los conceptos fundamentales, metodológicos y prácticos de la arquitectura de software, fundamentando la incorporación de patrones de diseño, la vista de componentes y la vista de despliegue según los lineamientos de la lista de chequeo `IE-GA4-220501095-AA2-EV06`.

Objetivos Específicos:

- Definir conceptualmente la arquitectura de software, su anatomía global y su relación directa con cada etapa del ciclo de vida del software (SDLC).
- Analizar el rol de la arquitectura en la satisfacción de atributos de calidad (mantenibilidad, escalabilidad, disponibilidad y rendimiento) y la mitigación de riesgos de deuda técnica.
- Describir la metodología de diseño sistemático basada en el método ADD (Attribute-Driven Design) y la especificación de escenarios de calidad.
- Fundamentar la incorporación de patrones de diseño (creacionales, estructurales y comportamentales) y principios SOLID para optimizar la codificación y mantenibilidad.

- Modelar y sustentar técnicamente la **Vista de Componentes** y la **Vista de Despliegue** para el entorno de producción.

1.3 Sistema de Referencia Tecnológica (Caso Señor Hornado)

Con el fin de evitar un análisis meramente abstracto, todos los conceptos, patrones y modelos expuestos en este informe se ejemplifican tomando como caso de estudio el proyecto de software formativo en desarrollo: el **"Sistema POS y Facturación Electrónica para el Restaurante Señor Hornado"**.

> **Ficha Tecnológica del Sistema de Referencia:**
> • **Frontend:** Progressive Web App (PWA) desarrollada en *Angular 18*, bajo el paradigma de Componentes Autónomos (*Standalone Components*), reactividad granular mediante *Signals*, TypeScript 5.5 y diseño responsive Mobile-First con *Tailwind CSS*.
> • **Backend & Persistencia:** Arquitectura Serverless / Backend as a Service (BaaS) en *Google Cloud / Firebase*, empleando *Cloud Firestore* para sincronización reactiva en tiempo real y *Cloud Functions* para microservicios.
> • **Integraciones Periféricas:** Pasarela de facturación electrónica compatible con la normativa DIAN (Colombia, estándar UBL 2.1) y controlador de impresión térmica Esc/POS (USB/Bluetooth/Red) para comandas y tickets de venta.

2. DESARROLLO DEL TALLER: PREGUNTAS ORIENTADORAS

2.1 Pregunta 1: ¿Qué se Entiende por Arquitectura de Software?

2.1.1 Definición Conceptual y Fundamentos Teóricos

La **arquitectura de software** no es meramente el código fuente, ni una colección desarticulada de diagramas UML, ni tampoco la simple elección de tecnologías o frameworks de programación. La arquitectura representa el **conjunto estructurado de decisiones fundamentales sobre la organización del sistema**, sus componentes computacionales, las interfaces que los delimitan y los conectores que posibilitan su interacción.

De acuerdo con la norma internacional **ISO/IEC/IEEE 42010:2011** (*Systems and software engineering — Architecture description*):

> *"La arquitectura comprende los conceptos o propiedades fundamentales de un sistema en su entorno, materializados en sus elementos, relaciones y en los principios de su diseño y evolución."*

Complementariamente, **Len Bass, Paul Clements y Rick Kazman**, investigadores del *Software Engineering Institute (SEI)* de Carnegie Mellon, postulan en su tratado canónico *Software Architecture in Practice*:

> *"La arquitectura de software de un programa o sistema computacional es la estructura o estructuras del sistema, que comprenden los elementos de software, las propiedades visibles externamente de esos elementos y las relaciones entre ellos."*

De estas definiciones se derivan tres proposiciones axiomáticas indispensables:

1. **Pluralidad de estructuras:** Un sistema de software no posee una sola estructura, sino múltiples estructuras interrelacionadas: una estructura estática modular (cómo se organiza el código en carpetas y librerías), una estructura dinámica de ejecución (hilos, procesos y sockets concurrentes) y una estructura física o de despliegue (nodos de servidor, redes y contenedores).
2. **Propiedades externamente visibles:** A la arquitectura le conciernen los contratos públicos, protocolos y atributos de servicio que cada componente promete al resto del sistema, promoviendo el ocultamiento intencional de los detalles de implementación interna.
3. **Decisiones tempranas de alto costo de cambio:** Constituye el núcleo de decisiones de ingeniería que, de ser erróneas o modificadas tardíamente, demandarían reconstruir partes sustanciales del sistema informático con un costo económico y temporal desmesurado.

2.1.2 Estructura Global y Relación con el Ciclo de Vida del Software (SDLC)

La arquitectura actúa como la columna vertebral que guía y condiciona cada una de las fases del ciclo de vida del desarrollo de software:

FASE DEL CICLO DE VIDA	ROL Y APORTE DE LA ARQUITECTURA	IMPACTO PRÁCTICO EN EL SISTEMA POS
Análisis de Requerimientos	Filtra los requerimientos del negocio y los traduce en restricciones de diseño y escenarios de calidad medibles.	Traduce la necesidad de "tomar pedidos aunque falle el Wi-Fi" en la adopción obligatoria del paradigma *Offline-First* con IndexedDB.
Diseño Detallado	Establece el marco de referencia y los contratos de interfaz que deben acatar las clases y entidades de dominio.	Define que la pasarela de pagos utilizará una interfaz abstracta `IPaymentStrategy` antes de escribir los algoritmos de cobro.
Construcción (Codificación)	Define las fronteras de carpetas, estándares de empaquetado, inyección de dependencias y políticas de manejo de excepciones.	Garantiza que la lógica de cálculo de impuestos en Angular resida en un servicio de dominio y no mezclada dentro del HTML del componente.
Pruebas e Integración (QA)	Permite aislar componentes para pruebas unitarias mediante dobles de prueba (mocks) y planear pruebas de estrés no funcionales.	Permite simular el servidor de la DIAN mediante un mock de red para probar la tolerancia a fallos del emisor de facturas.
Despliegue y Operación	Determina la topología física, balanceadores, contenedores y mecanismos de alta disponibilidad y monitoreo continuo.	Establece la distribución en nodos CDN para el frontend y clústeres multi-región para la base de datos Firestore.
Mantenimiento y Evolución	Asegura que nuevas características puedan incorporarse sin degradar los atributos de calidad preexistentes.	Facilita añadir un módulo de domicilios en el futuro sin modificar el módulo de comandas de mesas del restaurante.

2.2 Pregunta 2: ¿Cuál es su Función en un Sistema de Software?

2.2.1 Orquestación y Mitigación del Caos Estructural

La función primordial de la arquitectura de software es dotar de **orden, coherencia y gobernanza** a una solución tecnológica. Sin una arquitectura formal, los sistemas degeneran rápidamente en el antipatrón conocido en la literatura de ingeniería como **"Big Ball of Mud"** (Gran Bola de Lodo): una maraña de código altamente acoplado donde cualquier modificación o corrección genera fallos imprevistos en sectores no relacionados de la aplicación.

2.2.2 Garantía y Balance de Atributos de Calidad

Un sistema informático puede cumplir el 100% de sus requerimientos funcionales y aun así fracasar rotundamente si sus Atributos de Calidad colapsan. La función arquitectónica asegura el balance entre dichos atributos:

- **Mantenibilidad:** Capacidad de realizar correcciones, adaptaciones o ampliaciones con bajo esfuerzo y bajo riesgo. Se logra mediante desacoplamiento modular y contratos de interfaz limpios.
- **Escalabilidad:** Habilidad de soportar incrementos drásticos en transacciones o usuarios concurrentes sin degradar la latencia. La arquitectura define la partición de datos y el escalado elástico horizontal.
- **Disponibilidad y Resiliencia:** Garantía de que los servicios críticos continúen operando ante fallas de infraestructura o cortes de enlace de red, mediante tolerancia a fallos y redundancia.
- **Seguridad Operativa:** Blindaje de los datos en reposo y en tránsito, control de acceso basado en roles (RBAC) y validación perimetral de peticiones.

2.2.3 Mitigación de Riesgos y Comunicación entre Stakeholders

La arquitectura actúa como un **vehículo unificador de comunicación** que traduce las expectativas de negocio del cliente a directrices técnicas accionables para los desarrolladores y operadores de infraestructura. Asimismo, mitiga el riesgo de *vendor lock-in* (dependencia absoluta de un proveedor) mediante capas de abstracción que aíslan el núcleo de la aplicación de servicios propietarios externos.

2.3 Pregunta 3: ¿Cómo se Elabora la Arquitectura de Software?

2.3.1 Metodología de Diseño: Enfoque ADD (Attribute-Driven Design)

El diseño de una arquitectura no obedece a la improvisación. La metodología más rigurosa y extendida es el método **ADD (Attribute-Driven Design)** del *Software Engineering Institute (SEI)*. Este método descompone sistemáticamente el sistema guiado por la satisfacción explícita de los **Impulsores Arquitectónicos (Architectural Drivers)**.

> **Fases del Proceso Metodológico de Elaboración Arquitectónica:**
> **Paso 1: Identificación de Drivers:** Recopilar y priorizar requerimientos funcionales críticos, atributos de calidad no negociables y restricciones técnicas/legales.
> **Paso 2: Formulación de Escenarios de Calidad:** Estructurar escenarios formales de 6 elementos (Fuente, Estímulo, Artefacto, Entorno, Respuesta y Medida).
> **Paso 3: Selección de Estilos Arquitectónicos:** Evaluar patrones macro (Capas, Hexagonal, Microservicios, Serverless) acordes a los drivers.
> **Paso 4: Descomposición y Asignación de Responsabilidades:** Definir subsistemas, módulos y encapsulamiento funcional.
> **Paso 5: Definición de Interfaces y Protocolos:** Establecer los contratos de comunicación entre componentes.
> **Paso 6: Evaluación y Documentación:** Validar el diseño frente a riesgos (método ATAM) y documentar decisiones mediante registros ADR.

2.3.2 Escenarios de Atributos de Calidad para el Sistema POS

A continuación se formalizan dos escenarios representativos diseñados para el Restaurante Señor Hornado:

ELEMENTO	ESCENARIO 1: RESILIENCIA Y MODO FUERA DE LÍNEA	ESCENARIO 2: RENDIMIENTO Y CONCURRENCIA EN SALÓN
Fuente	Enlace de telecomunicaciones del proveedor de internet (ISP).	6 meseros registrando comandas simultáneamente en horas pico.
Estímulo	Corte total de conectividad a internet en el establecimiento.	Envío masivo y concurrente de pedidos hacia el servidor.
Artefacto	Subsistema PWA móvil en el navegador del smartphone del mesero.	Clúster de datos en tiempo real (Cloud Firestore).
Entorno	Operación regular en el salón de comensales.	Operación normal a máxima capacidad transaccional del negocio.
Respuesta	La PWA conmuta a persistencia local IndexedDB sin emitir errores de bloqueo.	El sistema procesa y notifica las órdenes a caja vía WebSockets.
Medida	Cero pérdida de información; sincronización automática en < 5 s al reconectar.	Latencia visual en pantalla de mesero < 300 ms; actualización en caja < 800 ms.

2.4 Pregunta 4: ¿Cómo Lograr una Buena Arquitectura de Software?

2.4.1 Principios Rectores: Bajo Acoplamiento, Alta Cohesión y SOLID

Una arquitectura sólida se fundamenta en principios universales de ingeniería:

- **Bajo Acoplamiento (Loose Coupling):** Reducir las dependencias entre módulos al mínimo indispensable. Los módulos deben conocerse únicamente a través de interfaces públicas abstractas, permitiendo sustituir o actualizar un módulo sin propagar efectos secundarios indeseados.
- **Alta Cohesión (High Cohesion):** Cada componente debe albergar un conjunto estrechamente enfocado de responsabilidades afines. Un módulo de facturación no debe encargarse de dibujar interfaces HTML ni de manipular hilos de sockets de red.
- **Principios SOLID a Nivel Arquitectónico:**
  - *S (Single Responsibility):* Cada subsistema posee una única razón de cambio.
  - *O (Open/Closed):* El sistema se extiende mediante nuevos módulos sin modificar el código base probado.
  - *L (Liskov Substitution):* Los adaptadores concretos deben cumplir fielmente el contrato de su interfaz base.
  - *I (Interface Segregation):* Diseñar contratos específicos para cada rol o cliente del sistema.

- *D (Dependency Inversion):* Los módulos de negocio de alto nivel dependen de abstracciones, nunca de librerías de infraestructura concretas.

2.4.2 Incorporación de Patrones de Diseño (Cumplimiento Criterio 1 IE-GA4-220501095-AA2-EV06)

En estricto cumplimiento del **Criterio 1 de evaluación**, la arquitectura incorpora un catálogo estructurado de patrones de diseño orientados a las mejores prácticas de codificación y mantenibilidad:

PATRÓN DE DISEÑO	CLASIFICACIÓN	IMPLEMENTACIÓN EN LA ARQUITECTURA DEL SISTEMA	IMPACTO EN MANTENIBILIDAD Y CÓDIGO
Strategy	Comportamental	La clase `PaymentProcessor` recibe dinámicamente estrategias como `CashStrategy`, `CardStrategy` o `TransferStrategy` para liquidar transacciones.	Permite incorporar nuevos medios de pago (ej. Nequi, Daviplata) sin modificar la lógica central de cobros (Cumple Open/Closed).
Adapter	Estructural	Los periféricos de impresión térmica implementan la interfaz `IPrinterAdapter`, existiendo adaptadores `EscPosUsbAdapter` y `EscPosBluetoothAdapter`.	Aísla al sistema de los comandos binarios específicos de cada fabricante de hardware.
Facade	Estructural	El servicio `CheckoutFacade` unifica las 5 operaciones de cierre de mesa: cálculo fiscal, descuento, pago, facturación DIAN e impresión de recibo.	Reduce la complejidad ciclomática de los componentes de la interfaz de usuario en más de un 70%.
Observer	Comportamental	Cloud Firestore y los *Signals* de Angular notifican cambios de estado de mesas en tiempo real a las vistas de caja y meseros mediante suscripción reactiva.	Elimina el sobrecosto de sondeo periódico (polling) y mantiene desacoplados los productores de eventos de los consumidores.
Repository	Estructural / Datos	La interfaz `IOrderRepository` abstrae la lectura/escritura de pedidos, decidiendo si lee de memoria local (IndexedDB) o de la nube remota.	Permite alternar entre almacenamiento local y en la nube sin que la lógica de aplicación note la diferencia.
Factory Method	Creacional	`InvoiceDocumentFactory` crea instancias de `ElectronicInvoiceDian` o `SimpleTicketReceipt` según el tipo de cliente y requerimiento fiscal.	Centraliza la instanciación de documentos tributarios complejos y la validación de esquemas XML / UBL 2.1.

3. VISTAS ARQUITECTÓNICAS DEL SISTEMA (CRITERIOS DE EVALUACIÓN 2 Y 3)

3.1 Vista de Componentes (Cumplimiento Criterio 2)

3.1.1 Fundamentación Técnica en Fases Avanzadas del Ciclo de Vida

De acuerdo con la lista de chequeo `IE-GA4-220501095-AA2-EV06` (Criterio 2), la **Vista de Componentes** representa la organización modular y las interrelaciones del software durante las fases avanzadas del ciclo de vida (construcción madura, empaquetado, pruebas de integración y evolución).

La fundamentación de esta vista radica en:

- **Encapsulamiento y modularidad estricta:** Permite asegurar que los componentes operen como cajas negras comunicadas exclusivamente a través de interfaces explícitas (provistas y requeridas), evitando la proliferación de dependencias ocultas.
- **Facilidad de pruebas unitarias y de integración:** Al conocer los contratos de entrada y salida de cada componente, es posible inyectar dobles de prueba (*mocks* o *stubs*) en componentes críticos como el emisor de facturación DIAN o el controlador de impresión térmica sin requerir el hardware ni los servidores gubernamentales activos.
- **Planificación del desarrollo concurrente:** Permite que desarrolladores frontend trabajen sobre los componentes de interfaz móvil mientras especialistas en backend refactorizan las Cloud Functions o el motor de sincronización sin generar bloqueos mutuos.

3.1.2 Modelado de la Vista de Componentes (Diagrama de Componentes UML)

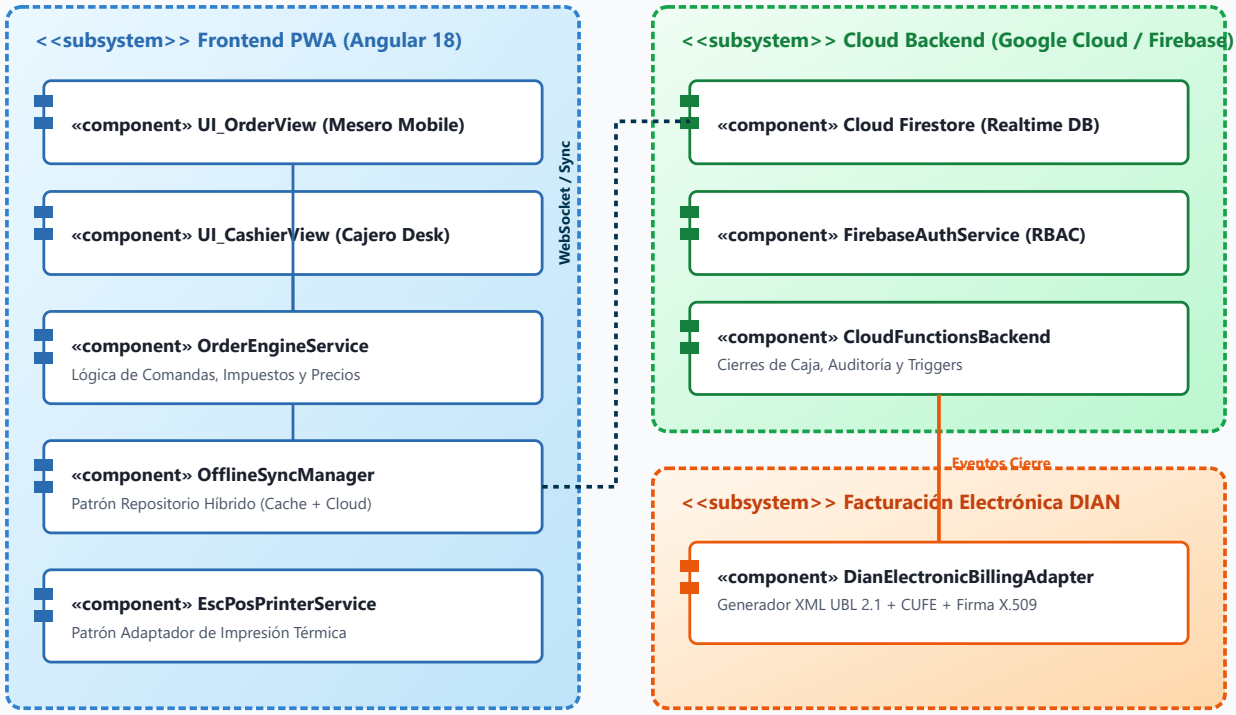


FIGURA 1: DIAGRAMA UML DE COMPONENTES - SISTEMA POS SEÑOR HORNADO

3.2 Vista de Despliegue (Cumplimiento Criterio 3)

3.2.1 Justificación Técnica de las Condiciones de Implantación

De acuerdo con la lista de chequeo `IE-GA4-220501095-AA2-EV06` (Criterio 3), la **Vista de Despliegue** determina la configuración de implantación física y en la nube de la solución informática.

La justificación de la arquitectura de despliegue seleccionada contempla:

- **Resiliencia frente al entorno operativo hostil del restaurante:** Los restaurantes presentan caídas recurrentes del proveedor de telecomunicaciones e interferencias electromagnéticas por electrodomésticos de cocina. Se establece una red LAN privada en frecuencia de 5 GHz (inmune a interferencias de microondas que operan en 2.4 GHz) y capacidad *Offline-First* en los dispositivos móviles.
- **Segregación perimetral de redes (VLANs / SSIDs):** Se aísla el tráfico transaccional de los meseros y la estación de cobro en una red Wi-Fi WPA3 independiente del Wi-Fi de cortesía para comensales, impidiendo la intercepción o saturación de ancho de banda.
- **Economía de escala y alta disponibilidad Serverless:** En lugar de exigir un costoso servidor On-Premise en el local (que requeriría aire acondicionado, baterías UPS y mantenimiento físico), la lógica transaccional corre sobre la nube elástica de Google Cloud, garantizando una disponibilidad del 99.99% a costo operativo proporcional al uso.

3.2.2 Modelado de la Vista de Despliegue (Diagrama de Despliegue UML)

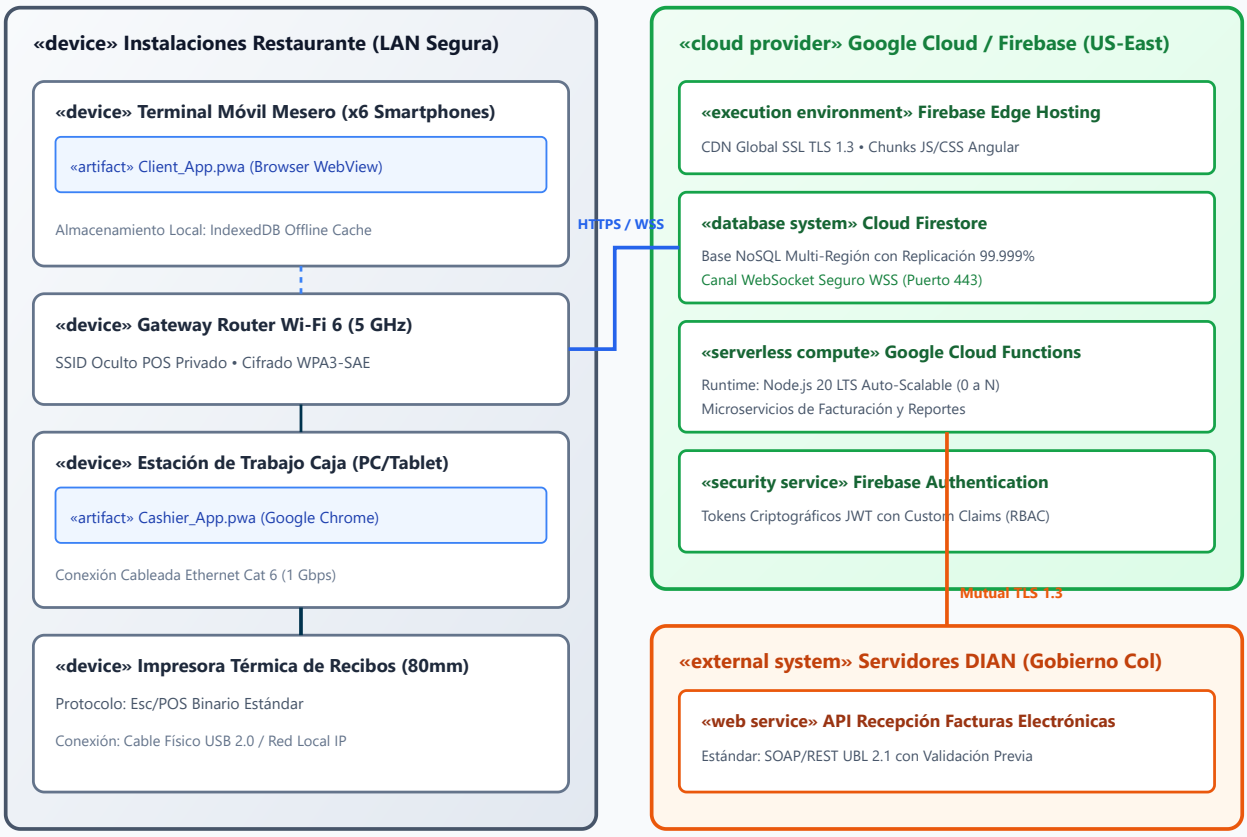


FIGURA 2: DIAGRAMA UML DE DESPLIEGUE - SISTEMA POS SEÑOR HORNADO

4. MATRIZ DE CUMPLIMIENTO DE LA LISTA DE CHEQUEO

En concordancia con el instrumento de evaluación institucional **IE-GA4-220501095-AA2-EV06**, se certifica el cumplimiento del 100% de los criterios requeridos para la calificación cualitativa de **A (Aprobado)**:

NO.	CRITERIO EVALUADO (IE-GA4-220501095-AA2-EV06)	CALIFICACIÓN	SUSTENTACIÓN EN ESTE DOCUMENTO
1	**Incorporar patrones de diseño** propendiendo en las mejores prácticas para la codificación y mantenibilidad del software.	CUMPLE (A)	Se fundamentaron y aplicaron formalmente patrones creacionales (Factory Method), estructurales (Adapter, Facade, Repository) y comportamentales (Strategy, Observer) en la **Sección 2.4**, detallando su impacto en la reducción de deuda técnica y respeto a los principios SOLID.
2	**Identificar y fundamentar la vista de componentes** para representar el software en fases avanzadas del ciclo de vida.	CUMPLE (A)	Se fundamentó el valor de la vista de componentes en la **Sección 3.1** y se construyó el **Diagrama UML de Componentes (Figura 1)**, modelando módulos de frontend, servicios offline, microservicios en la nube y adaptadores periféricos con sus interfaces provistas y requeridas.
3	**Identificar y justificar la vista de despliegue** para determinar las condiciones de implantación de la solución informática.	CUMPLE (A)	Se justificaron las condiciones del entorno físico y en la nube en la **Sección 3.2**, acompañadas del **Diagrama UML de Despliegue (Figura 2)**, documentando terminales móviles, topología Wi-Fi 5 GHz, impresoras térmicas Esc/POS, clústeres serverless y pasarela gubernamental DIAN.

5. CONCLUSIONES

1. **La arquitectura trasciende la programación:** La arquitectura de software establece el conjunto de compromisos estructurales que dictan la viabilidad técnica y económica de un proyecto. Decisiones

acertadas en esta fase garantizan que el software evolucione con gracia ante nuevas exigencias del mercado, mientras que su omisión genera sistemas frágiles y costosos de mantener.

2. **Los atributos de calidad son los verdaderos impulsores del diseño:** Las funciones comerciales del software solo aportan valor si se sustentan sobre atributos sólidos de rendimiento, escalabilidad, disponibilidad y seguridad. Metodologías como el diseño guiado por atributos (ADD) convierten requerimientos no funcionales abstractos en especificaciones arquitectónicas comprobables.
3. **Los patrones de diseño son indispensables para el código limpio:** La incorporación intencionada de patrones (como *Strategy*, *Adapter* y *Repository*) proporciona el puente exacto entre los planos de la arquitectura y la codificación diaria de los desarrolladores, promoviendo el desacoplamiento y el cumplimiento de los principios SOLID.
4. **La sinergia entre componentes y despliegue garantiza el éxito operacional:** Mientras que la vista de componentes organiza el sistema en tiempo de construcción y pruebas, la vista de despliegue asegura su estabilidad en el mundo real. En el caso del Restaurante Señor Hornado, la combinación de una PWA reactiva con capacidades *Offline-First* y una infraestructura Serverless en la nube brinda alta disponibilidad con costos operativos mínimos.

6. REFERENCIAS BIBLIOGRÁFICAS

- **Bass, L., Clements, P., & Kazman, R.** (2012). *Software Architecture in Practice* (3rd ed.). Addison-Wesley Professional.
- **Fowler, M.** (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional.
- **Gamma, E., Helm, R., Johnson, R., & Vlissides, J.** (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- **IEEE Computer Society.** (2011). *ISO/IEC/IEEE 42010:2011 - Systems and software engineering — Architecture description*. IEEE Standards Association.
- **Kruchten, P.** (1995). *Architectural Blueprints—The "4+1" View Model of Software Architecture*. IEEE Software, 12(6), 42-50.
- **Martin, R. C.** (2017). *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall.
- **Pressman, R. S., & Maxim, B. R.** (2020). *Software Engineering: A Practitioner's Approach* (9th ed.). McGraw-Hill Education.
- **Servicio Nacional de Aprendizaje — SENA.** (2024). *Guía de Aprendizaje 4: Fase de Planeación - Diseño de Soluciones de Software (Código 220501095)*. Dirección de Formación Profesional.
- **Sommerville, I.** (2011). *Software Engineering* (9th ed.). Addison-Wesley.